

Python

09

Лямбда функції

Lambda функція приймає будь-яку кількість аргументів (або жодного), але складається з одного виразу, який повертає функцією значення.

Значення, що повертається - значення, якому присвоєна функція.

`lambda parameters: expression`

```
1 >>list(map(lambda x: x*x , [4, 5, 6]))  
2 [16, 25, 36]
```

Коли використовувати Lambda, а коли ні

Якщо ви виявляєте, що намагаєтеся використати щось, що не підтримує лямбда-вираз, це, ймовірно, ознака того, що нормальна функція підійде краще.

Спроба подолати той факс, що лямбда-функція Пайтон не підтримує оператори є ще одним червоним прапором.

Рекурсія

Рекурсивна функція – це така функція, що викликає сама себе

```
def factorial_recursive(n):  
    if n == 1:  
        return n  
    else:  
        return n*factorial_recursive(n-1)
```

Первый вызов

```
factorial_recursive(3):  
    if 3 == 1:  
        return 3  
    else:  
        return 3*factorial_recursive(3-1)
```

Второй вызов

```
factorial_recursive(2):  
    if 2 == 1:  
        return 2  
    else:  
        return 2*factorial_recursive(2-1)
```

Третий вызов

```
factorial_recursive(1):  
    if 1 == 1:  
        return 1  
    else:  
        return 1*factorial_recursive(1-1)
```

Анотації

Анотації потрібні для того, щоб підвищити інформативність вашого коду.
Найзатребуванішою у цьому сенсі темою є контроль типів змінних

Строки документації

Строки документації - строкові літерали, які є першим оператором у модулі, функції, класі або визначення методу.

Така строка документації стає спеціальним атрибутом `__doc__` цього об'єкта.

Функції вищого порядку

Функція вищого порядку - це функція, яка може приймати як аргумент іншу функцію та/або повертати функцію як результат роботи

Map, filter. reduce

Функція **map()** використовується для застосування вхідної функції до кожного елемента об'єкта, що ітерується (наприклад списку або словника) і повернення нового ітератора для отримання результатів. Після роботи функції **map()** кількість елементів нового об'єкта буде такою самою, як у ітеріруемого об'єкта, але самі елементи будуть змінені роботою вхідної функції.

Функція **filter()** застосовує вхідну функцію до ітеріруемого об'єкта (список, строка і т.д.), перевіряючи чи потрібно зберегти конкретний елемент чи ні. Простими словами, вона відфільтровує все, частину або жодного елемента з об'єкта, що ітерується. Після роботи функції **filter()** кількість елементів нового об'єкта може бути рівною або меншою, ніж у ітеріруемого об'єкта, самі елементи при цьому не можуть бути якимось змінені.

Функція **reduce()** так само як і попередні дві функції приймає 2 аргументи - вхідну функцію та ітеріруемый об'єкт. Вона послідовно застосовує вхідну функцію до елементів ітеріруемого об'єкта разом з результатом попереднього застосування і повертає одиничне значення. Виходить як би накопичувальний результат.

Функція map()

```
result = map(lambda x: x**2, [1, 2, 3, 4, 5, 6])  
print(list(result))  
>>> [1, 4, 9, 16, 25, 36]
```

Функція filter()

```
result = filter(lambda x: x % 2 == 0, [1, 2, 3, 4, 5, 6])  
print(list(result))  
>>> [2, 4, 6]
```

Функція reduce()

```
from functools import reduce  
result = reduce(lambda a, x: a + x**2, [1, 2, 3], 0)  
>>> 14
```

3 map, filter, reduce

Пример	Определение fn	Output
<code>list(map(fn, [5, 6, 7]))</code>	<code>def fn(x): return x + 5</code>	<code>[10, 11, 12]</code>
<code>list(filter(fn, [1, 2, 6, 7]))</code>	<code>def fn(x): return x > 5</code>	<code>[6, 7]</code>
<code>reduce(fn, [2, 3], 5)</code>	<code>def fn(x, y): return x * y</code>	<code>30</code>